Spec — Gespräche: Server-Push Analysis via WebSocket

Created: 2026-06-18 — Claude.ai Status: Ready for `_working/` *Replaces:* `spec_gesprache_async_analysis_2026-06-18.md` *(Claude Code draft)*

Context

When the user taps [Analysieren →], the LLM call takes 10-20 seconds. The current implementation blocks the UI — the page is frozen until the response arrives. If the user switches tabs or closes the app, the in-flight request is lost.

Robert's requirements:

- Close the app, switch to Lesen or Curator — analysis still completes
- Start another session and analyse that too — parallel fine
- Results end up in Archiv automatically
- No polling — server pushes result to client when ready

Solution: server-side background thread + WebSocket push. No database queue. No Celery. No Redis. Python threading only. Flask-SocketIO for the WebSocket layer.

Architecture

```
Client                                Server
  |                                     |
  |-- WebSocket connect (on load) →     |
  |← socket.id assigned ------------- |
  |                                     |
  |-- POST /api/review/submit -----→  |
  |   {transcript, persona, model,    |
  |    scene, socket_id}              |
  |← {job_id} immediately ----------- |  (uuid4, no wait)
  |                                     |
  |  [user does anything —              |  [threading.Thread starts]
  |   switch tabs, new session,         |  [run_review() called]
  |   close app and reopen]             |  [no DB, in-memory only]
  |                                     |
  |                                     |  [analysis complete]
  |                                     |  [write to archive]
  |← WS emit: analysis_complete ----- |  [push result to socket_id]
```

```
|   {job_id, result, archive_id}    |
|                                   |
| update card, render feedback,     |
| show archive link                 |
```

Dependencies

One new dependency: `flask-socketio`

```bash
pip install flask-socketio
```

No queue, no broker, no additional services. `flask-socketio` uses `gevent` or `threading` mode — use `threading` mode to match the existing Flask setup.

Security

Three layers, all required before this goes to production.

Layer 1 — Encryption (WSS)

`app.minimoi.ai` already has HTTPS via Let's Encrypt. WebSocket connections over HTTPS automatically use WSS (WebSocket Secure) — encrypted end to end. Nothing to build in the app; handled by Nginx and the SSL certificate.

Required Nginx config for WebSocket (add to AWS Nginx config):

```nginx
location /socket.io/ {
    proxy_pass http://localhost:5001;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Without `Upgrade` and `Connection` headers, the WebSocket handshake fails silently and falls back to polling. This is a required change in the AWS Phase 2 Nginx config (already noted in `docs/AWS_MIGRATION_PLAN.md`).

Layer 2 — Authentication

Reject WebSocket connections from unauthenticated users:

```python
from flask_login import current_user

@socketio.on('connect')
def on_connect():
    if not current_user.is_authenticated:
        return False  # connection rejected before any data flows
    # Log connection for operational visibility
    app.logger.info(f"WebSocket connected — socket_id={request.sid} "
                    f"user={current_user.id}")
```

If someone connects without being logged in, the connection is refused immediately. No data flows, no socket_id assigned.

Layer 3 — Authorization (result isolation)

Analysis results are emitted to a specific `socket_id`, not broadcast. Each client only receives their own results:

```python
socketio.emit('analysis_complete', {...}, room=socket_id)
```

`room=socket_id` means the message goes to exactly one connected client. Even if multiple users were connected simultaneously, each only receives their own analysis results.

Security summary

Concern	Solution	Where
Data in transit	WSS (automatic with HTTPS)	Nginx + Let's Encrypt
Unauthenticated access	Reject on connect	Flask-SocketIO `on_connect`
Result leakage	Emit to specific socket_id only	`room=socket_id` in emit

Backend changes

1. Initialize SocketIO

In `app.py` (or portal init):

```python
from flask_socketio import SocketIO, emit

socketio = SocketIO(app, async_mode='threading', cors_allowed_origins='*')
```

Replace `app.run()` with `socketio.run(app, ...)` in the entry point.

2. In-memory job registry

```python
import threading
import uuid

# In-memory only — gone on restart, acceptable at this scale
_analysis_jobs = {}  # {job_id: {status, socket_id, session_id}}
_jobs_lock = threading.Lock()
```

3. Submit endpoint

```python
```

```python
@app.route('/api/review/submit', methods=['POST'])
def submit_review():
    data = request.json
    transcript = data['transcript']
    persona = data['persona']
    scene = data.get('scene', '')
    model = data.get('model', 'grok')
    socket_id = data['socket_id']
    session_id = data.get('session_id')

    job_id = str(uuid.uuid4())

    with _jobs_lock:
        _analysis_jobs[job_id] = {
            'status': 'running',
            'socket_id': socket_id,
            'session_id': session_id
        }

    # Fire background thread — no await, returns immediately
    thread = threading.Thread(
        target=_run_analysis,
        args=(job_id, transcript, persona, scene, model, socket_id,
              session_id),
        daemon=True
    )
    thread.start()

    return jsonify({'job_id': job_id})
```

4. Background analysis function

python

```
def _run_analysis(job_id, transcript, persona, scene, model,
                  socket_id, session_id):
    try:
        # Existing review call — unchanged
        result = run_review(transcript, persona, scene, model)

        # Write to archive
        archive_id = _write_to_archive(session_id, transcript, result)

        # Push result to client via WebSocket
        socketio.emit('analysis_complete', {
            'job_id': job_id,
            'result': result,
            'archive_id': archive_id
        }, room=socket_id)

    except Exception as e:
        socketio.emit('analysis_error', {
            'job_id': job_id,
            'error': str(e)
        }, room=socket_id)

    except Exception as e:
        # Log with full context for debugging / CloudWatch
        app.logger.error(
            f"Analysis failed — job_id={job_id} persona={persona} "
            f"model={model} error={type(e).__name__}: {e}"
        )
        socketio.emit('analysis_error', {
            'job_id': job_id,
            'error': str(e)
        }, room=socket_id)

    finally:
        # Always clean up in-memory registry
        with _jobs_lock:
            _analysis_jobs.pop(job_id, None)
```

5. Job watchdog — catch silent thread failures

A lightweight watchdog runs as a daemon thread on app startup. Every 30 seconds it scans `_analysis_jobs` for jobs older than 60 seconds. If found, the job has silently failed — emit error to client and clean up.

```python
import time
from datetime import datetime, timedelta

def _watchdog():
    while True:
        time.sleep(30)
        now = datetime.utcnow()
        stale = []
        with _jobs_lock:
            for job_id, job in _analysis_jobs.items():
                age = now - job['started_at']
                if age > timedelta(seconds=60):
                    stale.append((job_id, job['socket_id']))

        for job_id, socket_id in stale:
            app.logger.warning(
                f"Analysis job timed out — job_id={job_id}"
            )
            socketio.emit('analysis_error', {
                'job_id': job_id,
                'error': 'timeout'
            }, room=socket_id)
            with _jobs_lock:
                _analysis_jobs.pop(job_id, None)

# Start watchdog on app init
threading.Thread(target=_watchdog, daemon=True).start()
```

Add `started_at: datetime.utcnow()` to the job dict when created in the submit endpoint.

6. Archive write (side effect of completion)

```python
def _write_to_archive(session_id, transcript, result):
    # Write to existing archive table / file
    # Returns archive_id for the client to link to
    # Use existing archive write mechanism — no new schema needed
    pass  # implement using current archive pattern
```

6. WebSocket room per client

Flask-SocketIO automatically assigns each connected client a `socket.id`. Use this as the room — `emit(..., room=socket_id)` sends only to that client. No broadcast, no shared rooms.

Frontend changes

1. Connect WebSocket on page load

javascript

```javascript
const socket = io();  // connects to same server

socket.on('connect', () => {
    window.socketId = socket.id;  // store for use in submit
});

socket.on('analysis_complete', (data) => {
    const { job_id, result, archive_id } = data;
    _handleAnalysisComplete(job_id, result, archive_id);
});

socket.on('analysis_error', (data) => {
    const { job_id, error } = data;
    _handleAnalysisError(job_id);
});
```

2. Submit handler (replaces current Analysieren click)

javascript

```
async function submitAnalysis(session, card) {
    const btn = card.querySelector('.btn-session-card-analyse');
    btn.disabled = true;
    btn.textContent = 'Analysiere…';

    // Animated progress labels while waiting
    const labels = ['Analysiere…', 'Analysiere… Grammatik',
        'Analysiere… Wortschatz', 'Analysiere… Aussprache',
        'Analysiere… Zusammenfassung'];
    let i = 0;
    const ticker = setInterval(() => {
        btn.textContent = labels[++i % labels.length];
    }, 3000);

    const res = await fetch('/api/review/submit', {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({
            transcript: session.transcript,
            persona: session.persona,
            scene: session.scene,
            model: session.model,
            socket_id: window.socketId,
            session_id: session.id
        })
    });

    const { job_id } = await res.json();

    // Store ticker ref so completion handler can clear it
    card.dataset.jobId = job_id;
    card.dataset.tickerRef = ticker;

    // POST returns immediately — page is now fully free
}

function _handleAnalysisComplete(job_id, result, archive_id) {
    // Find card by job_id
    const card = document.querySelector(`[data-job-id="${job_id}"]`);
    if (!card) return;  // user navigated away — result goes to archive silentl

    clearInterval(parseInt(card.dataset.tickerRef));
    clearTimeout(parseInt(card.dataset.clientTimeout));
```

```
    // Render feedback below card
    renderFeedback(card, result);

    // Update button
    const btn = card.querySelector('.btn-session-card-analyse');
    btn.textContent = '✓ Analysiert';
    btn.disabled = true;

    // Show archive link
    if (archive_id) {
        const link = document.createElement('a');
        link.href = `/german/archiv/${archive_id}`;
        link.textContent = 'Im Archiv anzeigen →';
        link.className = 'archive-link';
        card.appendChild(link);
    }
}

function _handleAnalysisError(job_id) {
    const card = document.querySelector(`[data-job-id="${job_id}"]`);
    if (!card) return;

    clearInterval(parseInt(card.dataset.tickerRef));

    const btn = card.querySelector('.btn-session-card-analyse');
    btn.textContent = 'Fehler — Nochmal →';
    btn.disabled = false;
    // Re-attach click handler for retry
}
```

3. If user navigates away

If the card is not in the DOM when `analysis_complete` fires, `_handleAnalysisComplete` finds no card and returns silently. The result is already written to the archive by the server. When the user opens Archiv, the session is there. No result is lost. No error shown.

What does NOT change

- `run_review()` in `providers/review_router.py` — unchanged

- Feedback output format (FEEDBACK/FEHLER/STÄRKEN/SCHWÄCHEN) — unchanged
- The archive schema — write uses existing archive pattern
- Any other German page or route

Deployment note (for AWS migration)

Flask-SocketIO with `async_mode='threading'` works with gunicorn using the `--worker-class=geventwebsocket.gunicorn.workers.GeventWebSocketWorker` flag, or with `eventlet`. This needs to be configured correctly when the app moves to AWS EC2.

For local dev: `socketio.run(app, debug=True)` — unchanged from current `app.run()`.

Add to `docs/AWS_MIGRATION_PLAN.md` open questions: confirm WebSocket support in the Nginx config (needs `proxy_http_version 1.1` and `Upgrade`/`Connection` headers).

Error visibility and cleanup

Three failure scenarios and how each is handled:

Scenario	Detection	Client sees	Cleanup
LLM API error	Exception caught in thread	"Fehler — Nochmal →" immediately	`finally` block removes job
Silent thread death	Watchdog after 60s	"Fehler — Nochmal →" after ~60s	Watchdog removes job
Client timeout (WS drop)	Client 90s timer	"Fehler — Nochmal →" after 90s	Job removed by watchdog
Client navigates away	`analysis_complete` finds no card	Nothing (silent)	Archive has result

Logging: Every failure logs to Flask logger with job_id, persona, model, error type. On AWS this lands in CloudWatch. Alert on `Analysis failed` log entries to get notified of systematic failures (e.g. model provider down, rate limits hit).

No manual cleanup needed: The watchdog and `finally` blocks ensure `_analysis_jobs` never grows unbounded. In normal operation it contains only actively running jobs (0-3 entries at any time).

Definition of Done

- Tapping [Analysieren →] returns immediately — card shows "Analysiere…" and page is fully interactive
- Animated progress labels cycle while analysis runs
- User can switch to Lesen, Curator, start a new session — analysis continues on server
- Two cards can be analysed in parallel — each resolves independently
- Result pushes to client via WebSocket when ready
- Result written to Archiv automatically on completion
- If user has navigated away: result in Archiv, no card update, no error
- LLM API error: "Fehler — Nochmal →" appears immediately on card
- Silent thread failure: watchdog fires within 60s, card shows error
- Client 90s timeout: card shows error if no WS event in 90s
- All failures logged with job_id, persona, model, error type
- Watchdog running on app start — confirm in logs on startup
- Archive link appears on card after analysis: "Im Archiv anzeigen →"
- No regression on existing analysis output format
- Verified: start analysis → switch to Lesen → return to Gespräche → result is on card and in Archiv
- Verified: two parallel analyses complete independently
- Security verified:
  - WSS confirmed in browser dev tools (Network → WS → connection shows wss:// not ws://)
  - Unauthenticated WebSocket connection rejected (test in incognito)
  - Results only appear on the correct client (not broadcast)
- Nginx WebSocket config in place (Upgrade + Connection headers)

Commit

```
Gespräche: server-push analysis via WebSocket — non-blocking, persistent results, auto-archive on completion.
```

Spec · 2026-06-18 · Claude.ai Replaces: spec_gesprache_async_analysis_2026-06-18.md